

7. QUEUE ADTS

Queue concepts

Queue applications

A queue ADT: requirements, contract

Implementations of queues: using arrays
and linked-lists

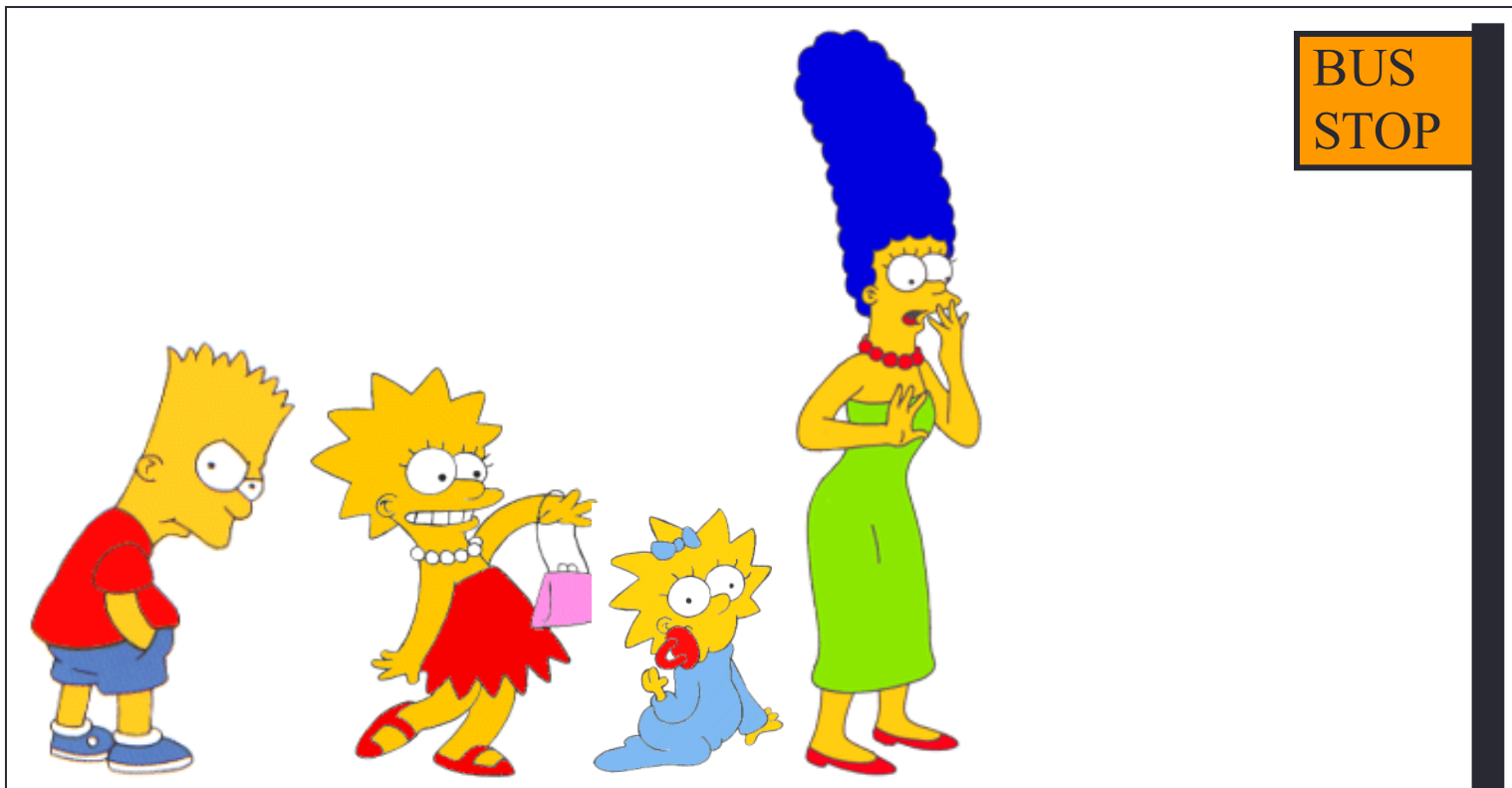
Queues in the Java class library

Queue concepts

- A **queue** is a first-in-first-out sequence of elements. Elements can be added only at one end (the **rear** of the queue) and removed only at the other end (the **front** of the queue).
- The **size** (or **length**) of a queue is the number of elements it contains.

Example: bus queue

- Consider a queue of people at a bus-stop:



Queue applications

- Print server

- Uses a queue of print jobs.

- Operating system

- Disk driver uses a queue of disk input/output requests.

- Scheduler uses a queue of processes awaiting a slice of processor time.

Example: demerging (1)

- Consider a file of person records, each of which contains a person's name, gender, birth-date, etc. The records are sorted by birth-date. We are required to rearrange the records such that females precede males, but they remain sorted by birth-date within each gender group.
- Bad idea: use a sorting algorithm.
Time complexity is $O(n \log n)$ at best.
- Good idea: use a **demerging algorithm**.
Time complexity is $O(n)$.

Example: demerging (2)

- Demerging algorithm:

To copy a file of person records from *input* to *output*, rearranged such that females precede males, but their order is otherwise unchanged:

1. Make empty queues *females* and *males*.
2. For each person *p* in *input*, repeat:
 - 2.1. If *p* is female, add *p* at the rear of *females*.
 - 2.2. If *p* is male, add *p* at the rear of *males*.
3. While *females* is not empty, repeat:
 - 3.1. Remove a person *f* from the front of *females*.
 - 3.2. Write *f* to *output*.
4. While *males* is not empty, repeat:
 - 4.1. Remove a person *m* from the front of *males*.
 - 4.2. Write *m* to *output*.
5. Terminate.

Queue ADT: requirements

- Requirements:
 - 1) It must be possible to make a queue empty.
 - 2) It must be possible to test whether a queue is empty.
 - 3) It must be possible to obtain the size of a queue.
 - 4) It must be possible to add an element at the rear of a queue.
 - 5) It must be possible to remove the front element from a queue.
 - 6) It must be possible to access the front element in a queue without removing it.

Queue ADT: contract (1)

- Possible contract for homogeneous queues (expressed as a Java generic interface):

```
public interface Queue<E> {  
    // Each Queue<E> object is a homogeneous queue  
    // whose elements are of type E.  
    ////////////////////////////////// Accessors //////////////////////////////////  
    public boolean isEmpty ();  
    // Return true if and only if this queue is empty.  
    public int size ();  
    // Return this queue's size.  
    public E getFirst ();  
    // Return the element at the front of this queue.
```

Queue ADT: contract (2)

- Possible contract (*continued*):

```
//////////////////// Transformers //////////////////////
```

```
public void clear ();
```

```
// Make this queue empty.
```

```
public void addLast (E it);
```

```
// Add it as the rear element of this queue.
```

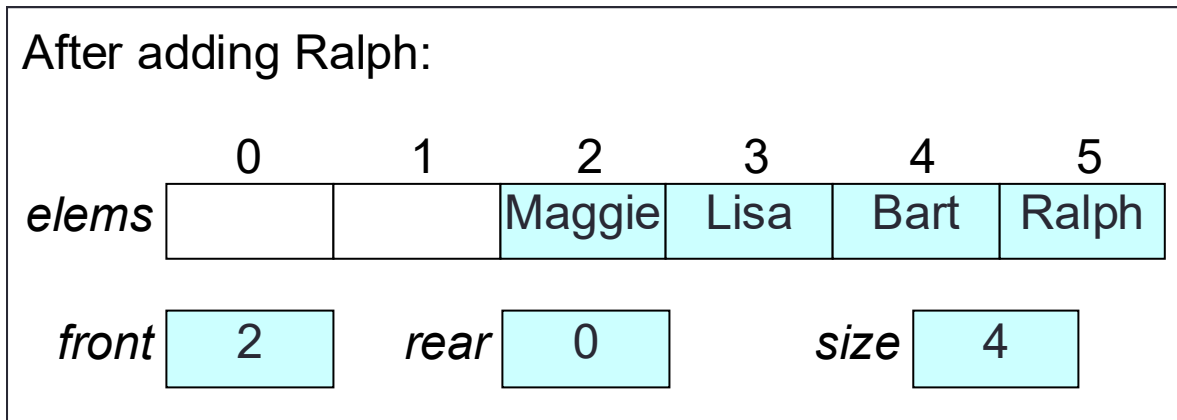
```
public E removeFirst ();
```

```
// Remove and return the front element of this queue.
```

```
}
```


Implementation of queues using arrays (2)

- Animation (with *cap* = 6):

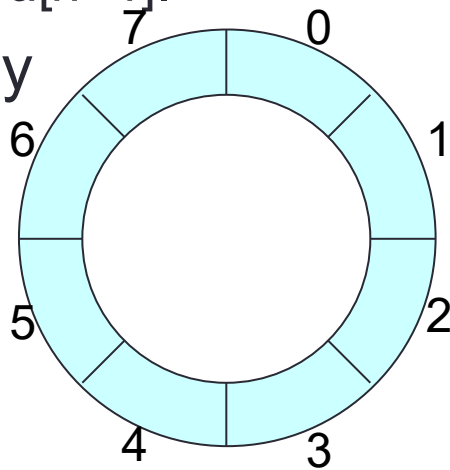


Implementation of queues using arrays (3)

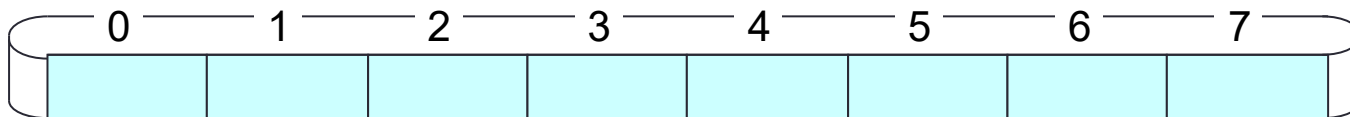
- Once the rightmost array slot is occupied, no more elements can be added, unless we shift elements to fill up any unoccupied leftmost slots.
- But then operation `addLast` would have time complexity $O(n)$, rather than $O(1)$.
- We can avoid this if we use a “cyclic array” instead of an ordinary array.

Cyclic arrays

- In a **cyclic array** a of length n , every slot has both a successor and a predecessor. In particular:
 - the successor of $a[n-1]$ is $a[0]$
 - the predecessor of $a[0]$ is $a[n-1]$.
- Visualizing a cyclic array (of length 8):

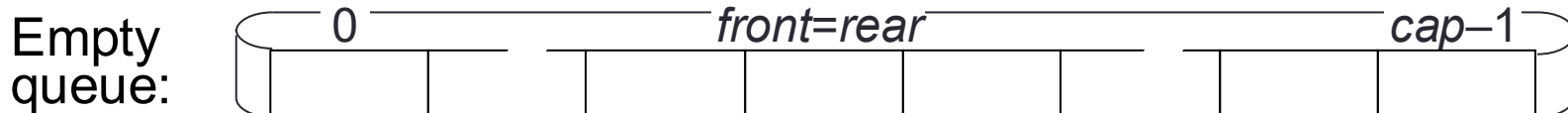
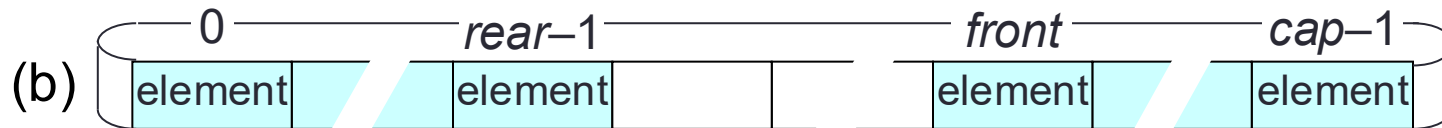
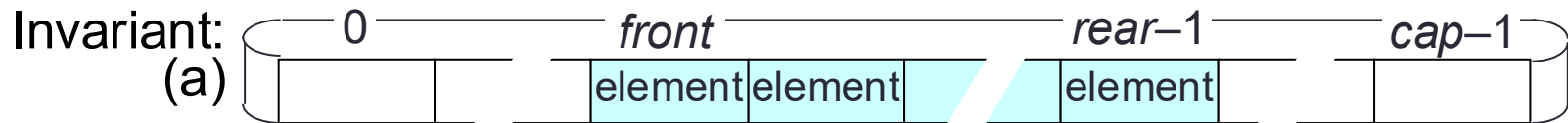


or



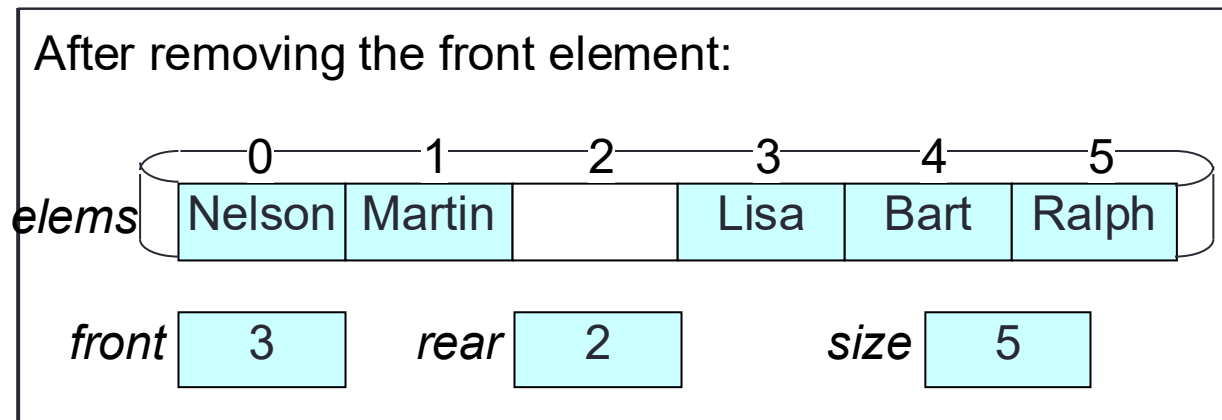
Implementation of queues using cyclic arrays (1)

- Represent a *bounded* queue (size $\leq cap$) by:
 - variables *size*, *front*, *rear*
 - a cyclic array *elems* of length *cap*, containing the elements either (a) in *elems*[*front*...*rear*-1] or (b) in *elems*[*front*...*cap*-1] and *elems*[0...*rear*-1].



Implementation of queues using cyclic arrays (2)

- Animation (with $cap = 6$):



Implementation of queues using cyclic arrays (3)

- Java implementation:

```
public class ArrayQueue<E>
    implements Queue<E> {
    private E[] elems;
    private int size, front, rear;
    ////////////////////////////////// Constructor //////////////////////////////////
    public ArrayQueue (int cap) {
        elems = (E[]) new Object[cap];
        size = front = rear = 0;
    }
```

Implementation of queues using cyclic arrays (4)

- Java implementation (*continued*):

```
//////////////////// Accessors //////////////////////  
public boolean isEmpty () {  
    return (size == 0);  
}  
  
public int size () {  
    return size;  
}  
  
public E getFirst () {  
    if (size == 0) throw ...;  
    return elems[front];  
}
```

Implementation of queues using cyclic arrays (5)

- Java implementation (*continued*):

```
//////////////////// Transformers //////////////////////  
public void clear () {  
    size = front = rear = 0;  
}  
  
public void addLast (E it) {  
    if (size == elems.length) ...  
    elems[rear++] = it;  
    if (rear == elems.length) rear = 0;  
    size++;  
}
```

---- NB

Implementation of queues using cyclic arrays (6)

- Java implementation (*continued*):

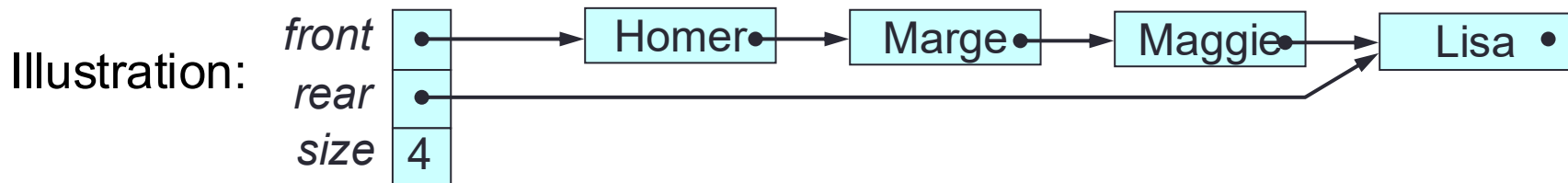
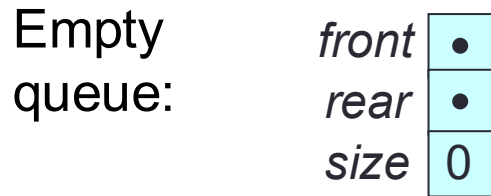
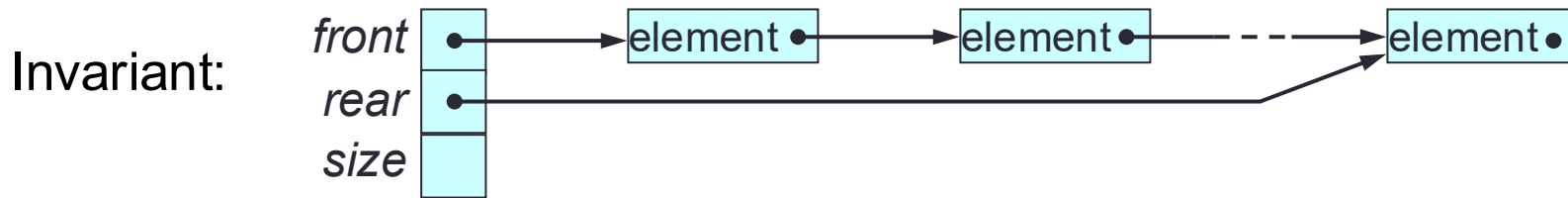
```
public E removeFirst () {  
    if (size == 0) throw ...;  
    E frontElem = elems[front];  
    elems[front++] = null;  
    if (front == elems.length) front = 0;  
    size--;  
    return frontElem;  
}
```

NB

- Analysis:
 - All operations have time complexity $O(1)$.

Implementation of queues using SLLs (1)

- Represent an (unbounded) queue by:
 - an SLL, whose header contains links to the first node (*front*) and last node (*rear*).
 - a variable *size* (optional).



Implementation of queues using SLLs (2)

- Java implementation:

```
public class LinkedListQueue<E>
    implements Queue<E> {
    private Node<E> front, rear;
    private int size;
    ////////////////////////////////// Inner class //////////////////////////////////
    private class Node<E> {
        public E element;
        public Node<E> next;

        public Node (E x, Node<E> s) {
            element = x; next = s;
        }
    }
}
```

Implementation of queues using SLLs (3)

- Java implementation (*continued*):

```
//////////////////// Constructor //////////////////////  
public LinkedList () {  
    front = rear = null;  
    size = 0;  
}  
  
//////////////////// Accessors //////////////////////  
public boolean isEmpty () {  
    return (front == null);  
}
```

Implementation of queues using SLLs (4)

- Java implementation (*continued*):

```
public int size () {  
    return size;  
}  
  
public E getFirst () {  
    if (front == null) throw ...;  
    return front.element;  
}
```

Implementation of queues using SLLs (5)

- Java implementation (*continued*):

```
//////////////////// Transformers //////////////////////  
public void clear () {  
    front = rear = null;  
    size = 0;  
}  
  
public void addLast (E it) {  
    Node<E> newest = new Node(it, null);  
    if (rear != null) rear.next = newest;  
    else front = newest;  
    rear = newest;  
    size++;  
}
```

Implementation of queues using SLLs (6)

- Java implementation (*continued*):

```
public E removeFirst () {  
    if (front == null) throw ...;  
    E frontElem = front.element;  
    front = front.next;  
    if (front == null) rear = null;  
    size--;  
    return frontElem;  
}  
}
```

- Analysis:

- All operations have time complexity $O(1)$.

Queues in the Java class library

- The library interface `java.util.Queue<E>` is similar to our interface `Queue<E>`.
- The library class `java.util.LinkedList<E>` implements `java.util.Queue<E>`, representing each queue by a *doubly-linked-list*. (This is overkill!)
- Illustration:

```
import java.util.*;
Queue<Person> busQ =
    new LinkedList<Person>();
busQ.addLast(homer);  busQ.addLast(marge);
busQ.addLast(maggie); busQ.addLast(lisa);
busQ.addLast(bart);
Person p = busQ.removeFirst();
```

Examples: demerging again (1)

1. Implementation of the demerging algorithm:
 - see Moodle

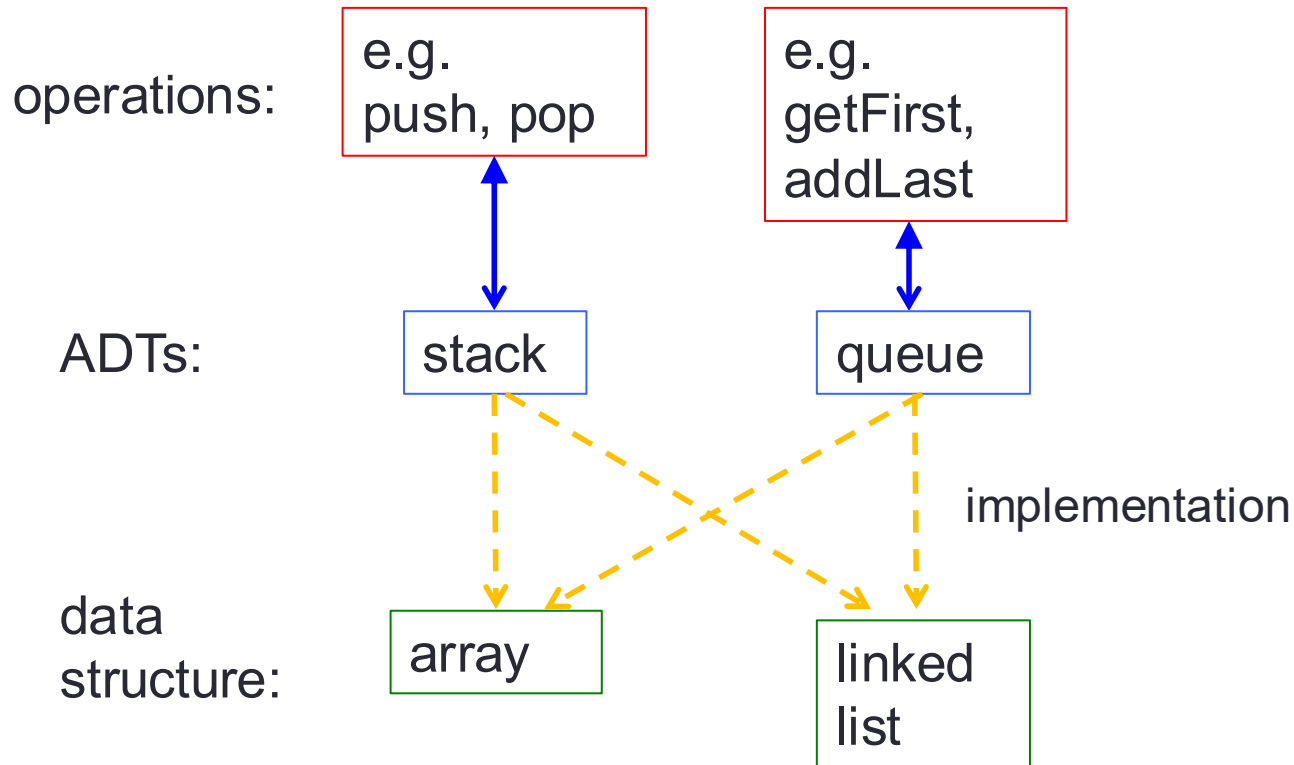
Examples: Deque

- 2. A ***doubly-ended queue*** (or ***deque***) allows elements to be added or removed at both ends of the queue. In other words, it supports the following operations in addition to the usual queue operations: $d.\text{addFirst}(x)$ adds x at the front of deque d ; $d.\text{removeLast}()$ removes the element at the rear of deque d ; $d.\text{getLast}()$ retrieves the element at the rear of deque d .

Write a deque contract in the form of a Java interface named Deque. Write a linked implementation of a deque as a Java class LinkedDeque. Make sure that all deque operations have time complexity $O(1)$.

See demo – how to start this

Summary of terms



An ADT algorithm is linked to one thing from each row (an ADT, an operation, and the data structure used to implement the ADT)